Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-380: $4^{η}$ Σειρά Ασκήσεων

• **Πρόβλημα $1^{ο}$:**

α) Αλγόριθμος πολυπλοκότητας $O(n^3)$:

∗ Χρησιμοποιούμε τρεις εμφωλευμένους βρόχους, ο πρώτος διατρέχει τον πίνακα A, ο δεύτερος τον B και ο τρίτος τον C - με δεικτες i, j και k αντίστοιχα. Αν βρούμε τρεις αριθμούς, έναν από κάθε πίνακα, που να αθροίζουν σε μηδέν ($A[i] + B[j] + C[k] == 0$), επιστρέφουμε True. Αφού ο κάθε βρόχος θα ελέξει όλα τα n στοιχεία, η πολυπλοκότητα θα είναι $O(n^3)$.

β) Αλγόριθμος πολυπλοκότητας $O(n^2 \log n)$:

∗ Αρχικά ταξινομούμε τον ένα από τους τρείς πίνακες, με οποιοδήποτε αλγόριθμο ο οποίος έχει πολυπλοκότητα $O(n \log n)$ - έστω τον C. Έπειτα, χρησιμοποιούμε δύο εμφωλευμένους βρόχους για να διατρέξουμε τους δύο μη-ταξινομημένους πίνακες, άρα η πολυπλοκότητα εδώ θα είναι $n \times n = n^2$ - έστω τους A και B, με a το τρέχον στοιχείο του A και b το τρέχον στοιχείο του B. Για κάθε επανάληψη, εφαρμόζουμε δυαδική αναζήτηση στον ταξινομημένο πίνακα, ψάχνοντας αν υπάρχει το στοιχείο $-(a+b)$ - απαιτειται $O(\log n)$ χρόνος. Τελικά, η συνολική πολυπλοκότητα είναι $O(n \log n) + n^2 \times \ O(\log n) = O(n^2 \log n)$.

γ) Αλγόριθμος πολυπλοκότητας $O(n^2)$:

∗ Ψευδοκώδικας - Αλγόριθμος Άθροισμα0 (A, B, C, n):
    // Ταξινόμηση των B και C σε αύξουσα σειρά ($O(nlogn)$):
        Ταξινόμησε(B); Ταξινόμησε(C);
        // Εξωτερικός βρόχος για τον πίνακα A (n επαναλήψεις):
        Για i από 0 έως $n-1$:
        // Ψάχνουμε δυο αριθμούς από τους B και C ώστε το άθροισμα τους να ισούτε με:
        $target = -A[i]$;
        // Αρχικοποίηση των δύο δεικτών που θα χρησιμοποιήσουμε:
        $j = 0$;
        $k = n - 1$;
        // Εσωτερική επανάληψη:
        Οσο $((j < n)$ και $(k >= 0))$:
            $sum = B[j] + C[k]$;
            Αν $(sum == target)$ τότε:
                // Ισχύει $A[i] + B[j] + C[k] = 0$:
                Επέστρεψε «Αληθές»;
            Αλλιώς_αν $(sum < target)$ τότε:
                // Το άθροισμα είναι μικρό, πρέπει να μεγαλώσει. Αφού B ταξινομημένος, αυξάνουμε τον δείκτη j:
                $j = j + 1$;
            Αλλιώς:
                // Το άθροισμα είναι μεγάλο, πρέπει να μικρύνει. Αφού C ταξινομημένος, μειώνουμε τον δείκτη k:
                $(k = k - 1)$;
    // Αν ελέξουμε ολόκληρο τον πινακα A και δεν έχουμε επιστρέψει:
    Επέστρεψε «Ψευδές»;
∗ ΤΕΛΟΣ

δ) Αλγόριθμος ταχύτερος από $O(n^2)$:

∗ « Το πρόβλημα πιστεύεται ότι δεν λύνεται ταχύτερα από $O(n^2)$ βάσει της Εικασίας 3SUM (3SUM Conjecture). »

• **Πρόβλημα 2ο:**

α) Αντεπαράδειγμα για τον Heaviest-First Greedy Algorithm:

∗ Έστω G = [2] − [3] − [2] ένα μονοπάτι ενός γράφου. Ο αλγόριθμος, για αυτό ως είσοδο, θα επιλέξει αρχικά τον κόμβο με το μεγαλύτερο βαρός, που είναι ο μεσαίος (3), και θα τον προσθέσει στο αρχικά κενό σύνολο S. Έτσι, αυτός και οι γείτονές του (2 και 2) θα διαγραφούν. Το γράφημα αδειάζει, άρα το αποτέλεσμα του αλγορίθμου είναι συνολικό βάρος 3. Όμως, η βέλτιστη λύση είναι να επιλέξουμε τους δύο ακραίους κόμβους (τα δύο 2άρια), οι οποίοι δεν συνδέονται μεταξύ τους. Σε αυτή την περίπτωση, το συνολικό τους βάρος είναι 2 + 2 = 4. Επειδή 4 > 3, αυτό σημαίνει πως ο άπληστος αλγόριθμος δεν βρίσκει πάντα ένα ανεξάρτητο σύνολο μέγιστου βάρους.

β) Αντιπαράδειγμα για τον αλγόριθμο Άρτιων/Περιττών:

∗ Έστω G = [2] − [1] − [1] − [2] ένα μονοπάτι ενός γράφου. Ο αλγόριθμος, για αυτό ως είσοδο, θα δημιουργήσει τα δύο σύνολα: S1 με τις θέσεις 1 και 3, άρα βάρη 2 και 1 που μας κάνει 3 & S2 με τις θέσεις 2 και 4, οπότε βάρη 1 και 2 που μας κάνει επίσης 3. Τελικά, επιστρέφεται ένα απο τα δύο σύνολα, και έχουμε συνολικό βάρος 3. Όμως, θα μπορούσαμε να επιλέξουμε τις θέσεις 1 και 4, και έτσι βάρη 2 και 2, με συνολικό βάρος 4, οι οποίοι αποτελούν ένα ανεξάρτητο σύνολο (αφού δεν ενώνονται). Επειδη 4 ’ 3, ο αλγόριθμος άρτιων/περιττών αποτυγχάνει.

γ) Πολυωνυμικός Αλγόριθμος:

∗ Ψευδοκώδικας - Αλγόριθμος ΜέγιστοΑνεξάρτητοΣύνολο (W, n):
// Είσοδος: W πίνακας βαρών μεγέθους n, όπου n ο αριθμός των κόμβων.
// Έξοδος: S το ανεξάρτητο σύνολο κόμβων με το μέγιστο συνολικό βάρος.
// Αρχικοποίηση:
Αν $n == 0$: Επέστρεψε {};
Αν $n == 1$: Επέστρεψε {ν_1};
// Πίνακας Α για αποθήκευση των μέγιστων βαρών ($n + 1$ δείκτες):
$A[0] = 0$; $A[1] = W[1]$;
// Γέμισμα πίνακα (δυναμικός προγραμματισμός):
Για $i = 2$ έως n: $A[i] = max(A[i-1], A[i-2] + W[i])$;
// Ανακατασκευή της λύσης - Αρχικοποίηση κενού συνόλου:
$S = \{\}$; $i = n$;
Όσο $i >= 1$:
// Ελέγχουμε ποια από τις δύο επιλογές μας έδωσε το μέγιστο βάρος στο βήμα i και την κρατάμε:
Αν $A[i] == A[i-1]$ τότε: $i = i - 1$;
Αλλιώς: $S.add(\nu_i)$; $i = i - 2$;
Επέστρεψε S;
∗ ΤΕΛΟΣ

δ) Άλλες Κατηγορίες Γράφων με Αποδοτικούς Αλγορίθμους:

∗ Δέντρα (Trees) ...

• **Πρόβλημα 3ο:**

Αλγόριθμος ΕλάχιστεςΚεραίες (σπίτια):
Αν (μέγεθος(σπίτια)) == 0 τότε: Επέστρεψε 0;
Ταξινόμηση(σπίτια);
εμβέλεια = 4; αριθμός_κεραιών = 0; τρέχον = 1; ν = μέγεθος(σπίτια);
Όσο (τρέχον <= ν):
πρώτο_ακάλυπτο = σπίτια[τρέχον];
θέση_κεραίας = πρώτο_ακάλυπτο + εμβέλεια;
αριθμός_κεραιών = αριθμός_κεραιών + 1;
Όσο (τρέχον <= ν) ΚΑΙ (σπίτια[τρέχον] <= θέση_κεραίας + εμβέλεια): τρέχον = τρέχον + 1;
Επίστρεψε (αριθμός_κεραιών);
Τέλος Αλγορίθμου.

• **Πρόβλημα 4ο:**

αi) Απόδειξη ότι πάντα επιστρέφει τοπικό ελάχιστο:

∗ Ο αλγόριθμος μετακινείται από τον τρέχοντα κόμβο v σε έναν γείτονα u μόνο εάν $x_u < x_v$. Αυτό σημανει ότι σε κάθε βήμα η τιμή του τρέχον κόμβου μειώνεται. Επειδή το πλέγμα έχει πεπερασμένο αριθμό κόμβων (n^2), το πλήθος των βημάτων είναι πεπερασμένο και ο αλγόριθμος είναι εγγυημένο ότι θα τερματίσει. Εκείνη τη στιγμή η παραπάνω συνθήκη παύει να ισχύει, άρα δεν υπάρχει κανένας γείτονας u του τρέχοντος κόμβου v που να έχει τιμή μικρότερη από το x_v. Αφού όλοι οι αριθμοί στο πλέγμα είναι μοναδικοί, ο κόμβος v είναι τοπικο ελάχιστο του γράφου G.

αii) Χρονική πολυπλοκότητα στη χείριστη περίπτωση:

∗ Στη χείριστη περίπτωση, οι τιμές των κόμβων είναι διατεταγμένες με τέτοιο τρόπο ώστε το μονοπάτι των φθινουσών τιμών να διασχίζει όλους τους κόμβους του πλέγματος πριν καταλήξει στο τοπικό ελάχιστο. Ο μέγιστος αριθμός κόμβων που μπορεί να επισκεφτεί είναι n^2. Σε κάθε βήμα, ο έλεγχος των γειτόνων απαιτεί σταθερό χρόνο $O(1)$, αφού κάθε κόμβος έχει το πολύ 4 γείτονες. Επομένως, η χρονική πολυπλοκότητα στη χείριστη περίπτωση είναι $\Theta(n^2)$.

β_1) Περιγραφή (Ιδέα) του Αλγόριθμου $O(n)$:

0. Είσοδος − Έξοδος: ενα $n \times n$ πλέγμα (γράφο) G, όπου κάθε κόμβος προσδιορίζεται από ένα ζεύγος συντεταγμένων (i, j) με $1 \leq i \leq n$ και $1 \leq j \leq n$ και έχει για τιμή ένα μοναδικό φυσικό αριθμό $x_{i,j}$ − ένας συγκεκριμένος κόμβος $v = (i, j)$ όπου θα είναι εγγυημένα ένας από τους ελάχιστους.

1. Βάση: αν το πλέγμα ειναι πολύ μικρό (π.χ. 1×1 ή 2×2), βρίσκουμε το τοπικό ελάχιστο ελέγχοντας όλους τους κόμβους.

2. Διαίρεση: θεωρούμε τους κόμβους που ανήκουν στην περίμετρο ($top : i = 0$, $bottom : i = n-1$, $left : j = 0$, $right : j = n-1$) του τρέχοντος πλέγματος, καθώς και τη μεσαία γραμμή ($i = \lfloor n/2 \rfloor$) και τη μεσαία στήλη ($j = \lfloor n/2 \rfloor$). Αυτοι οι κόμβοι χωρίζουν το εσωτερικό του πλέγματος σε 4 υπο-πλέγματα (τεταρτημόρια) μεγέθους περίπου $n/2 \times n/2$ (αφού $n = 2^k$ άρτιο).

3. Διαίρει και Βασίλευε (αναδρομή): εντοπίζουμε τον κόμβο με την ελάχιστη τιμή x_m ανάμεσα στους κόμβους της περιμέτρου και του σταυρού, και ελεγχουμε τους γείτονές του. Αν το x_m είναι μικρότερο από όλους τους γείτονές του, επιστρέφουμε τον κόμβο m ως τοπικό ελάχιστο. Διαφορετικά, υπάρχει ένας γείτονας u με $x_u < x_m$. Συνεχίζουμε αναδρομικά τον αλγόριθμο μόνο στο τεταρτημόριο που περιέχει τον κόμβο u.

∗ Σημείωση: για να λειτουργήσει η αναδρομή, ο αλγόριθμος θα χρειαστεί ως επιπλέον είσοδο τα όρια του τρέχοντος υπο-πλέγματος που εξετάζει κάθε φορά, δηλαδή τη γραμμή/στήλη εκκίνησης και τη γραμμή/στήλη τερματισμού. Αρχικά θα είναι από 1 έως n ...

β_2) Απόδειξη Ορθότητας:

∗ Ο αλγόριθμος κατευθύνεται σε ένα τεταρτημόριο μονο εάν υπάρχει κόμβος u εντός αυτού, με $x_u < x_m$, όπου x_m είναι η ελάχιστη τιμή στα σύνορα αυτού του τεταρτημορίου. Επειδή τα σύνορα έχουν τιμές μεγαλύτερες του x_u, οποιοδήποτε φθίνον μονοπάτι (που αναζητά τοπικό ελάχιστο) ξεκινήσει από το u, δεν μπορεί να δραπετεύσει από τα όρια του τεταρτημορίου. Συνεπώς, το υπο-πλέγμα αυτό είναι εγγυημένο ότι περιέχει τουλάχιστον ένα τοπικό ελάχιστο του αρχικού γράφου G. Αφού σε κάθε βήμα το μέγεθος του πλέγματος μειώνεται και είναι πεπερασμένο, ο αλγόριθμος θα τερματίσει και θα επιστρέψει ένα έγκυρο τοπικό ελάχιστο.

β_3) Πολυπλοκότητα:

∗ Σε ένα πλέγμα $n \times n$, το πληθος των στοιχείων της περιμέτρου και του κεντρικού σταυρού είναι ανάλογο του n (συγκεκριμένα $6n - 5$ στοιχεία). Ο εντοπισμός του ελαχίστου x_m απαιτεί έτσι χρόνο $O(n)$. Ο έλεγχος των γειτόνων παίρνει σταθερό χρόνο $O(1)$. Στη συνέχεια, η αναδρομή καλείται σε ένα υπο-πλέγμα μεγέθους $n/2 \times n/2$. Η αναδρομική σχέση για τον χρονο εκτέλεσης θα ειναι: $T(n) = 1 * T(n/2) + O(n)$. Οπότε, με βάση το Master Theorem (και συγκεκριμένα την τρίτη περίπτωση του), αυτή η αναδρομική εξίσωση έχει λύση: $T(n) = O(n)$ (ή Θ(n)).

• **Πρόβλημα 5ο:**

⋆ Ο Αλγόριθμος (Ψευδοκώδικας):

∗ Σημείωση: ο συγκεκριμένος αλγόριθμος δημιουργήθηκε με τη βοήθεια τεχνητής νοημοσύνης. ∗

$FFT3(a)$:
    $n = length(a)$;
    $if\ (n == 1) :\ return\ a$;
    $a_0\ =\ [\ a[0],\ a[3],\ a[6],\ ...,\ a[n-3]\ ]$;
    $a_1\ =\ [\ a[1],\ a[4],\ a[7],\ ...,\ a[n-2]\ ]$;
    $a_2\ =\ [\ a[2],\ a[5],\ a[8],\ ...,\ a[n-1]\ ]$;
    $y_0\ =\ FFT3(a_0)$;
    $y_1\ =\ FFT3(a_1)$;
    $y_2\ =\ FFT3(a_2)$;
    $y[n]$;
    $m = n/3$;
    $omega = 1$;
    $omega3 = e^{(2*\pi*i/3)}$;
    $omegaN = e^{(2*\pi*i/n)}$;
    $for\ (k = 0)\ to\ (m-1)$:
        $t_1\ =\ omega * y_1[k]$;
        $t_2\ =\ (omega^2) * y_2[k]$;
        $y[k] = y_0[k] + t_1 + t_2$;
        $y[k+m] = y_0[k] + t_1 * omega3 + t_2 * (omega3^2)$;
        $y[k+2*m] = y_0[k] + t_1 * (omega3^2) + t_2 * omega3$;
        $omega = omega * omegaN$;
    $return\ y$;

⋆ Αναδρομική Σχέση - Κεντρικό Θεώρημα:

Ο αλγοριθμος χωρίζει το πρόβλημα μεγέθους n σε 3 υποπροβλήματα, το καθένα με μέγεθος $n/3$. Επιπλέον, ο συνολικός χρόνος για τον διαχωρισμό του διανύσματος είναι $n \times \Theta(1)$ (ανάγνωση και τοποθέτησή όλων των στοιχείων του επί τις μαθηματικές πράξεις για τις $n/3$ επαναλήψεις του βρόχου), οπότε $\Theta(n)$. Τελικά: $T(n) = 3T\left(\frac{n}{3}\right) + \Theta(n)$. Σύμφωνα με το κεντρικό θεώρημα - $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ - έχουμε $a = 3,\ b = 3,\ f(n) = \Theta(n)$, και αρα $n^{\log_b a} = n^{\log_3 3} = n^1 = n$. Είμαστε στην δεύτερη περίπτωση του θεωρήματος, όπου $f(n) = \Theta(n^{\log_b a})$ και $T(n) = \Theta(n^{\log_b a} \log n)$. Τελικά: $\underline{T(n) = \Theta(n \log n)}$.

• **Πρόβλημα 6ο:**

∗ Ιδέα: ο αλγόριθμος είναι βασισμένος στον αλγόριθμο «Αναζήτησης Κατά Πλάτος - BFS» ...

Αλγόριθμος Αριθμός_Συντομότερων_Μονοπατιών (G, v, u):
    // G: Ο μη κατευθυνόμενος γράφος (V, E) & v: Ο κόμβος αφετηρία & u: Ο κόμβος προορισμός.
    For every node x in V: // 1. Αρχικοποίηση...
        $dist[x] = \infty$; $paths[x] = 0$; // Απόσταση από τον v (αρχικά άπειρο) & Αριθμός μονοπατιών (αρχικά 0).
    $dist[v] = 0$; $paths[v] = 1$; // Η απόσταση του v από τον εαυτό του είναι 0 & Υπάρχει 1 κενό μονοπάτι για τον εαυτό του.
    $Q = new_queue()$; $Q.enqueue(v)$; // Δημιουργούμε μια κενή ουρά και βάζουμε τον αρχικό κόμβο σε αυτή.
    While Q is not empty: // 2. Εκτέλεση Τροποποιημένης BFS...
        $curr = Q.dequeue()$; // Βγάζουμε τον πρώτο κόμβο από την ουρά.
        For every neighbour nxt of curr: // Ελέγχουμε όλους τους γείτονες του τρέχοντος κόμβου.
            $If\ dist[nxt] == \infty$: // Περίπτωση 1: τον επισκεπτόμαστε για πρώτη φορά.
                $dist[nxt] = dist[curr] + 1$; // Ενημερώνουμε την ελάχιστη απόστασή του.
                $paths[nxt] = paths[curr]$; // Μεταφέρουμε τον αριθμό μονοπατιών του $curr$ στον nxt.
                $Q.enqueue(nxt)$; // Εισάγουμε τον nxt στην ουρά για μελλοντική εξερεύνηση.
            $Else\ if\ dist[nxt] == dist[curr] + 1$: // Περίπτωση 2: Τον έχουμε ξαναδεί κ' βρήκαμε άλλο ένα σύντομο μονοπάτι.
                $paths[nxt] = paths[nxt] + paths[curr]$; // Προσθέτουμε τα νέα μονοπάτια στα ήδη γνωστά.
    Return $paths[u]$; // 3. Αποτέλεσμα...

∗ Πολυπλοκότητα:

Κάθε κόμβος (από τους n) μπαίνει και βγαίνει από την ουρά ακριβώς μία φορά (οταν η απόστασή του αλλάζει από ∞ σε πραγματικό αριθμό), άρα συνολικά απαιτείται χρόνος $O(n)$. Κάθε ακμή (από τις m) εξετάζεται ακριβώς δύο φορές (αφου ο γράφος είναι μη κατευθυνόμενος), άρα έχουμε $2m$ προσπελάσεις, γεγονός που μεταφράζεται σε χρόνο $O(m)$. Οι λειτουργίες ελέγχου και πρόσθεσης μέσα στον βρόχο απαιτούν σταθερό χρόνο $O(1)$ ανά προσπέλαση. Τελικά, ο συνολικός χρόνος εκτέλεσης προκύπτει από το άθροισμα αυτών: $O(n) + O(m) = O(n + m)$ ...

• **Πρόβλημα 7ο:** ∗ Δεν κατάφερα να λύσω το συγκεκριμενο πρόβλημα ...

• **Πρόβλημα 8ο:**

– Πρόβλημα8 (Λίστα Καλεσμένοι, Λίστα Ζευγάρια):

```
Πίνακας Κατάλληλο[Μέγεθος(Καλεσμένοι)] ⟵ Άδειος
Πίνακας Γνωστοί[Μέγεθος(Καλεσμένοι)] ⟵ Άδειος
Ουρά Ακατάλληλοι ⟵ Κενη
Για κάθε Άτομο στην λίστα Καλεσμένοι:
    Γνωστοί[Άτομο] ⟵ Πλήθος γνωστών από λίστα Ζευγάρια
    Κατάλληλο[Άτομο] ⟵ Αληθές
Για κάθε Άτομο στην λίστα Καλεσμένοι:
    Αν (Γνωστοί[Άτομο] < 5) ή (Γνωστοί[Άτομο] > Μέγεθος(Καλεσμένοι) − 6) τότε:
        Πρόσθεσε το Άτομο στην ουρά Ακατάλληλοι
        Κατάλληλο[Άτομο] ⟵ Ψευδές
Όσο η ουρά Ακατάλληλοι δεν είναι κενή:
    Άτομο ⟵ Αφαίρεσε το πρώτο στοιχείο από την ουρά Ακατάλληλοι
    Αφαίρεσε το Άτομο από τη λίστα Καλεσμένοι
    Για κάθε Άλλος στην λίστα Καλεσμένοι:
        Αν ο Άλλος είναι γνωστός του Άτομο τότε:
            Γνωστοι[Άλλος] ⟵ Γνωστοί[Άλλος] − 1
            Αν (Γνωστοί[Άλλος] < 5) και (Κατάλληλο[Άλλος] == Αληθές) τότε:
                Πρόσθεσε το Άλλος στην λίστα Ακατάλληλοι
                Κατάλληλο[Άλλος] ⟵ Ψευδές
        Αλλιώς:
            Αν (Γνωστοί[Άλλος] > Μέγεθος(Καλεσμένοι) − 6) και (Κατάλληλο[Άλλος] == Αληθές) τότε:
                Πρόσθεσε το Άλλος στην ουρα Ακατάλληλοι
                Κατάλληλο[Άλλος] ⟵ Ψευδές
    Επίστρεψε την λίστα Καλεσμένοι
```

– Πολυπλοκότητα:

Αρχικά, η πολυπλοκότητα του αλγορίθμου εξαρτάται από τον τρόπο που θα διαχειριστούμε την είσοδο «λίστα των ζευγαριών που γνωρίζονται μεταξύ», αλλά κατα μέσο όρο θα είναι $\mathcal{O}(n^2)$. Αυτό συμβαίνει αφού: α. αρχικοποίηση: ο υπολογισμός των αρχικών γνωριμιών για κάθε άτομο (ανάγνωση των ζευγαριών) και ο αρχικός έλεγχος καταλληλότητας απαιτεί, στη χειρότερη περίπτωση, τετραγωνικό χρόνο (όλα μέχρι πριν την δομή επανάληψης όσο). β. ουρά: κάθε άτομο, από τα n συνολικά, ελέγχεται μέσω του πίνακα ώστε να τοποθετηθεί στην ουρά το πολύ μία φορά. Επομένως, ο βρόχος όσο θα εκτελεστεί το πολύ n φορές. γ. ενημερωσεις: κάθε φορά που ένα άτομο εξάγεται από την ουρά και διαγράφεται, ο εσωτερικός βρόχος διατρέχει τα υπόλοιπα (το πολύ $n-1$) άτομα που βρίσκονται στη λίστα. Για κάθε ένα από αυτά τα άτομα, εκτελείται ένας σταθερός αριθμός πράξεων ελέγχου και μείωσης βαθμού. Έτσι, και η κύρια δομή επανάληψης χρειάζεται τετραγωνικό χρονο. Τελικά, ο αλγόριθμος εκτελειται σε $\mathcal{O}(n^2)$ ως προς το n.

• **Πρόβλημα 9ο:**

α) Αλγόριθμος για Διχρωματισμό ...

i) Ιδέα/Λογική αλγορίθμου: Έστω ένας γράφος $G = (V, E)$, όπου V το σύνολο των κόμβων και E το σύνολο των ακμών. Επιπλέον, έστω δύο χρώματα που συμβολίζουμε ως 1 και 2. Το 0 θα ειναι κατάσταση αχρωμάτιστος. Ένας αλγόριθμος που αποφασίζει αν ένας γράφος δέχεται έναν χρωματισμό με δυο χρώματα είναι ο εξής:

1. Αρχικοποιούμε όλους τους κόμβους του γράφου ως αχρωμάτιστους (χρώμα 0).
2. Επειδή ο γράφος μπορεί να αποτελείται από ξεχωριστά κομμάτια, επαναλαμβάνουμε τα παρακάτω για κάθε αχρωμάτιστο κόμβο:
3. Επιλέγουμε τον αχρωμάτιστο κόμβο s, του δίνουμε το χρώμα 1 και τον εισάγουμε σε μια ουρά.
4. Όσο η ουρά δεν είναι άδεια:
5. Εξάγουμε τον πρώτο κόμβο u από την ουρά.
6. Για κάθε γειτονικό κόμβο v του u:
7. Αν ο v είναι αχρωμάτιστος (χρώμα 0), του αναθέτουμε το αντίθετο χρώμα από τον u, και τον εισάγουμε στην ουρά.
8. Αν ο v είναι ήδη χρωματισμένος και έχει το ίδιο χρώμα με τον u, ο αλγόριθμος τερματίζει και επιστρέφει ΨΕΥΔΕΣ.
9. Αν χρωματιστούν ολοι οι κόμβοι χωρίς να προκύψει καμία σύγκρουση, ο αλγόριθμος επιστρέφει ΑΛΗΘΕΣ.

ii) Απόδειξη Ορθότητας: Υπάρχει ένα θεώρημα που σχετίζεται με τους γράφους και δηλώνει «ένας γράφος είναι διμερής (και έτσι χρωματίσιμος με 2 χρώματα) αν και μόνο αν δεν περιέχει κύκλους περιττού μήκους». Ο συγκεκριμένος αλγόριθμος, αν επιστρέψει ΨΕΥΔΕΣ σημαινει πως εντόπισε μια ακμή (u, v) όπου οι κόμβοι u και v που ενώνει, έχουν το ίδιο χρώμα και άρα δημιουργούν έναν κύκλο στον γράφο. Έτσι, σε αυτή την περίπτωση είναι αδύνατο να χρωματιστεί με δύο διαφορετικά χρώματα. Από την άλλη, αν επιστρέψει ΑΛΗΘΕΣ, σημαίνει ότι κάθε ακμή (u, v) στον γράφο συνδέει κόμβους με διαφορετικά χρώματα (αφού όλοι οι γείτονες ελέγχθηκαν και κανείς δεν βρέθηκε με το ίδιο χρώμα), και επομένως ο χρωματισμός με δύο χρώματα είναι έγκυρος.

iii) Πολυπλοκότητα: Αρχικά, για την χρονική πολυπλοκότητα, κάθε κόμβος $v \in V$ εισάγεται και εξάγεται από την ουρά το πολύ μία φορά - αυτό κοστίζει $O(V)$, και κάθε ακμή $(u, v) \in E$ ελέγχεται το πολύ δυο φορές (αν ο γράφος είναι μη-κατευθυνόμενος) - αυτό κοστίζει $O(E)$. Τελικά, ο συνολικός χρόνος εκτέλεσης είναι $O(V + E)$. Επιπλέον, για την χωρική πολυπλοκοτητα, απαιτείται ένας πίνακας μεγέθους $O(V)$ για την αποθήκευση των χρωμάτων και επιπλέον μνήμη $O(V)$ για την ουρά. Έτσι, η συνολική χωρική πολυπλοκότητα είναι $O(V)$.

β) Η ιδιότητα για διχρωματισμό και η απόδειξη ...

i) Όπως αναφέρθηκε προηγουμένος, «ενας γράφος δέχεται διχρωματισμό αν και μόνο αν δεν περιέχει κύκλους περιττού μήκους».

ii) Απόδειξη:

* Σημείωση: η συγκεκριμένη απόδειξη αντιγράφηκε από το διαδίκτυο *

« Απόδειξη. Έστω ένας μη κατευθυνόμενος γράφος $G = (V, E)$. Θα αποδείξουμε ότι ο G δέχεται διχρωματισμό (είναι διμερής) αν και μόνο αν δεν περιέχει κύκλους περιττού μήκους. Πρώτον, για το ευθύ, υποθέτουμε ότι ο γράφος είναι διμερής, δηλαδή οι κόμβοι του χωρίζονται σε δύο ξένα σύνολα (έστω Χρώμα 1 και Χρώμα 2) ώστε κάθε ακμή να συνδέει κόμβους διαφορετικού χρώματος. Αν υπήρχε ένας κύκλος $v_1, v_2, \ldots, v_k, v_1$, ξεκινώντας με τον v_1 στο Χρώμα 1, τα χρώματα θα εναλλάσσονταν υποχρεωτικά κατά μήκος του κύκλου. Έτσι, κάθε κόμβος v_i με περιττό δείκτη i θα είχε το Χρώμα 1 και με άρτιο το Χρώμα 2. Για να κλείσει ο κύκλος, ο τελευταίος κόμβος v_k πρέπει να συνδέεται με τον v_1 (που έχει το Χρώμα 1), άρα ο v_k πρέπει να έχει το Χρώμα 2, πράγμα που σημαίνει ότι ο δείκτης k πρέπει να είναι άρτιος. Επομένως, όλοι οι κύκλοι του γράφου έχουν υποχρεωτικά άρτιο μήκος. Αντίστροφα, υποθέτουμε τώρα ότι ο γράφος δεν έχει κύκλους περιττού μήκους. Χωρίς βλάβη της γενικότητας, έστω ότι είναι συνεκτικός. Επιλέγουμε έναν τυχαίο κόμβο u και τον χρωματίζουμε με το Χρώμα 1. Για κάθε άλλο κόμβο v, βρίσκουμε τη μικρότερη απόστασή του $d(u, v)$ από τον u. Αν η απόσταση είναι άρτια τον χρωματίζουμε με το Χρώμα 1, ενώ αν είναι περιττή του δίνουμε το Χρώμα 2. Αν αυτός ο χρωματισμός δεν ήταν έγκυρος, θα υπήρχαν δύο γειτονικοί κόμβοι v και w με το ίδιο χρώμα, άρα οι αποστάσεις τους $d(u, v)$ και $d(u, w)$ θα είχαν την ίδια αρτιότητα. Αν z είναι ο τελευταίος κοινός κόμβος στα συντομότερα μονοπάτια από τον u προς αυτούς, τότε προκύπτει ότι οι αποστάσεις $d(z, v)$ και $d(z, w)$ θα είχαν επίσης την ίδια αρτιότητα μεταξύ τους, οπότε το άθροισμά τους θα ήταν άρτιος αριθμός. Ο κύκλος που σχηματίζεται πηγαίνοντας από το z στο v, διασχίζοντας την ακμή (v, w) που έχει μήκος 1, και επιστρέφοντας στο z από το w, θα είχε συνολικό μήκος $d(z, v) + d(z, w) + 1$, το οποίο είναι υποχρεωτικά περιττός αριθμός. Αυτό όμως έρχεται σε άμεση αντίφαση με την υπόθεσή μας ότι δεν υπάρχουν κύκλοι περιττού μήκους (άτοπο), αποδεικνύοντας έτσι ότι κανένα ζευγάρι γειτονικών κόμβων δεν μπορεί να έχει το ίδιο χρώμα και συνεπώς ο γράφος δέχεται διχρωματισμό. Τέλος. »

γ) Το Graph Coloring ως Πρόβλημα Απόφασης και η κλάση NP ...

i) Περιγραφή: Το Graph Coloring Problem ως πρόβλημα απόφασης για k χρώματα διατυπώνεται ως εξής: « Για είσοδο έναν γράφο $G = (V, E)$ και ένα θετικό ακέραιο αριθμό k, υπάρχει συνάρτηση χρωματισμού $c : V \to \{1, 2, \ldots, k\}$ τέτοια ώστε για κάθε ακμή $(u, v) \in E$ να ισχύει $c(u) \neq c(v)$? ».

ii) ...

ii) Απόδειξη ότι ανήκει στην κλάση NP: Για να αποδείξουμε ότι το πρόβλημα απόφασης του k-χρωματισμού ανήκει στην κλάση NP, αρκεί να δείξουμε ότι κάθε θετικό στιγμιότυπο διαθέτει ένα πιστοποιητικό πολυωνυμικού μεγέθους το οποίο μπορεί να επαληθευτεί σε πολυωνυμικό χρόνο από έναν ντετερμινιστικό αλγόριθμο. Ως πιστοποιητικό ορίζουμε την ίδια την ανάθεση των χρωμάτων στους κόμβους του γράφου $G = (V, E)$, δηλαδή μια συνάρτηση $c : V \to \{1, 2, \ldots, k\}$, η οποία έχει μέγεθος $\mathcal{O}(|V|)$ και συνεπώς είναι πολυωνυμική ως προς το μέγεθος της εισόδου. Ο αλγόριθμος επαλήθευσης ελέγχει αρχικά αν το χρώμα $c(v)$ κάθε κόμβου $v \in V$ ανήκει στο επιτρεπτό σύνολο $\{1, \ldots, k\}$, απαιτώντας χρόνο $\mathcal{O}(|V|)$, και στη συνέχεια διατρέχει όλες τις ακμές $(u, v) \in E$ ελέγχοντας αν ισχύει $c(u) \neq c(v)$, κάτι που απαιτεί χρόνο $\mathcal{O}(|E|)$. Εφόσον ο συνολικός χρόνος επαλήθευσης είναι $\mathcal{O}(|V| + |E|)$, δηλαδή γραμμικός και άρα πολυωνυμικός ως προς το μέγεθος του γράφου, προκύπτει ότι το πρόβλημα ανήκει πράγματι στην κλάση NP.

δ) 3-Coloring: NP-complete ή όχι & ειδικές περιπτώσεις για 4 χρώματα ...

i) Ναι, «το πρόβλημα αν ένας γράφος δέχεται έναν χρωματισμό με 3 χρώματα (3-coloring)») είναι NP-complete. Ένα επιχειρημα είναι πως στο πρόβλημα των ερωτημάτων (α) και (β), οι επιλογές ήταν αναγκαστικές. Δηλαδή, αν επιλέγαμε το χρώμα 1 για έναν κόμβο, οι γείτονές του επρεπε αναγκαστικά να πάρουν το χρώμα 2. Εδώ όμως, αυτός ο περιορισμός δεν υπάρχει. Αν βάψουμε έναν κόμβο χρώμα 1, ο γείτονάς του μπορεί να βαφτεί είτε 2 είτε 3. Αυτή η ελευθερία δημιουργεί πολλαπλά μονοπάτια αποφάσεων σε κάθε βήμα του αλγορίθμου και μπορεί να χρειαστεί να γυρίσουμε προς τα πίσω. Έτσι, το συγκεκριμένο πρόβλημα είναι NP-complete.

ii) Τι συμβαίνει για 4 χρώματα: όπως και με τα τρία χρώματα, έτσι και με τα τέσσερα, το πρόβλημα εξακολουθεί να είναι NP-complete, για παρόμοιους λόγους. Παρ΄ όλ΄ αυτά, υπάρχει ένα θεώρημα για την ειδική περίπτωση όπου ο γράφος είναι επίπεδος, το εξής: «κάθε επίπεδος γράφος μπορεί να χρωματιστεί με 4 το πολύ χρώματα».

• **Πρόβλημα 10ο:**

(Α) Πόσους κόμβους και πόσες ακμές έχει ο γράφος L(G) ...

1. Πλήθος κόμβων: ο γράφος $L(G)$ έχει m κόμβους (δηλαδή ακριβώς όσοι είναι και οι ακμές του αρχικού γράφου G). Απόδειξη: ο αλγοριθμος αναφέρει ρητά: «Create a line digraph L(G) with every edge of G as a node.».

2. Πλήθος ακμών: ο γράφος $L(G)$ έχει $\sum_{v \in V} d_{in}(v) \cdot d_{out}(v)$ ακμές, όπου $d_{in}(v)$ είναι ο βαθμός εισόδου (πλήθος εισερχόμενων ακμών) και $d_{out}(v)$ είναι ο βαθμός εξόδου (πλήθος εξερχόμενων ακμών) του κόμβου v. Απόδειξη: εξ ορισμού στον νεο γράφο, υπάρχει μια ακμή από τον κόμβο e_1 στον κόμβο e_2 (του $L(G)$) αν και μόνο αν η ακμή e_1 του G καταλήγει στον ίδιο κόμβο v από τον οποίο ξεκινάει η ακμή e_2. Για οποιονδήποτε κόμβο v του G, κάθε μία από τις $d_{in}(v)$ εισερχόμενες ακμές πρέπει να συνδεθεί με κάθε μία από τις $d_{out}(v)$ εξερχόμενες ακμές. Επομένως, ο κόμβος v είναι υπευθύνος για τη δημιουργια $d_{in}(v) \cdot d_{out}(v)$ νέων ακμών στον $L(G)$. Το συνολικό πλήθος των ακμών του $L(G)$ είναι το άθροισμα αυτών των γινομένων για όλους τους κόμβους V του γράφου.

(Β) Σε ποιον γνωστό αλγόριθμο μοιάζει και γιατί ...

∗ Ο αλγόριθμος μοιάζει στον αλγόριθμο Αναζήτησης Κατά Βάθος (DFS). Αυτό φαίνεται επειδή η δομή της διαδικασίας μοιάζει με την λογική αυτού του αλγορίθμου. Πιο συγκεκριμένα, i. ξεκινάει από έναν κόμβο v και τον σημαδεύει ως επισκεμμένο, ii. προσπελάζει ολες τις εξερχόμενες ακμές (γείτονες) του v, iii. για κάθε γείτονα u, ελέγχει αν έχει ήδη επισκεφθεί, και αν δεν έχει επισκεφθεί, καλεί αναδρομικά τον εαυτό της για τον u, προχωρώντας πιο βαθιά στον γράφο, πριν οπισθοχωρήσει.

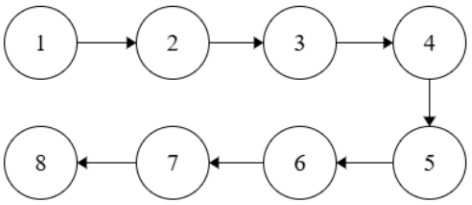
(Γ) Ποια είναι η χρονική πολυπλοκότητα του αλγορίθμου LineDigraph ...

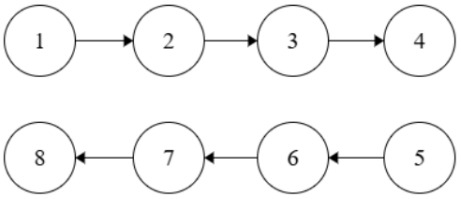
1. Πολυπλοκότητα: του αλγορίθμου είναι $O(n + m + m_L)$, όπου $n = |V|$, $m = |E|$ και m_L είναι το πλήθος των ακμών του τελικού Line Digraph.

2. Απόδειξη: ουσιαστικά, ο αλγόριθμος λειτουργεί ως μια παραλλαγή της αναζήτησης κατά βάθος (DFS). Η αρχική δημιουργία των κόμβων του νέου γράφου κοστίζει $O(m)$, ενώ η βασική διάσχιση απαιτεί χρόνο $\mathcal{O}(n + m)$ για να μαρκάρει τον κάθε κόμβο ως επισκεμμένο και να εξετάσει την κάθε ακμή του αρχικού γράφου. Το επιπλέον υπολογιστικό κόστος προκύπτει αποκλειστικά από τη δημιουργία των νέων συνδέσεων: για κάθε κόμβο u που επεξεργαζόμαστε, ο αλγόριθμος πρέπει να εννώσει την εισερχόμενη ακμή του με όλες τις εξερχόμενες. Αυτή η διαδικασία εκτελείται αθροιστικά για όλο τον γραφο ακριβώς τόσες φορές όσες είναι και οι νέες ακμές που δημιουργούνται στον $L(G)$ (δηλαδή $\sum d_{in} \cdot d_{out}$), προσθέτοντας $O(m_L)$ βήματα. Επομένως, συνδυάζοντας την αρχικοποίηση, τη δομική διάσχιση και τη δημιουργία των νέων ακμών, ο συνολικός απαιτούμενος χρόνος θα είναι $O(n + m + m_L)$.

• **Πρόβλημα 11ο:**

α) Σχήματα (δύο γράφοι) ...
∗ Αριστερά: κατευθυνόμενος άκυκλος γράφος με 8 κόμβους που δεν έχει κανένα μη συγκρίσιμο ζευγάρι κόμβων.
∗ Δεξιά: κατευθυνόμενος άκυκλος γράφος με 8 κόμβους που έχει τουλάχιστον ένα μη συγκρίσιμο ζευγάρι κόμβων.





β) Γραμμικός αλγόριθμος που αποφασίζει αν κατ. άκυκλος γράφος περιέχει τουλάχιστον ένα μη συγκρίσιμο ζευγάρι κόμβων ...
1: Εκτελούμε Αναζήτηση κατά Βάθος (DFS) στον γράφο.
2: Κάθε φορά που η αναζήτηση για έναν κόμβο ολοκληρώνεται, τον τοποθετούμε σε μια στοίβα.
3: Όταν τελειώσει η διαδικασία για όλους τους κόμβους, το άδειασμα της στοίβας μας δίνει μια ταξινομημενη διάταξη.
4: Έστω $v_1, v_2, v_3, \ldots, v_n$. Την διατρέχουμε από τον κόμβο v_1 έως τον v_{n-1}.
5: Για κάθε i από 1 έως $n-1$, ελέγχουμε αν υπάρχει κατευθυνόμενη ακμή από τον κόμβο v_i στον κόμβο v_{i+1}.
6: Αν για όλα τα i βρεθεί η ακμή $v_i \to v_{i+1}$, τότε έχουμε βρει ένα μονοπάτι που διασχίζει όλους τους κόμβους.
7: Αυτό σημαίνει ότι όλοι οι κόμβοι είναι συγκρίσιμοι μεταξύ τους. Ο αλγόριθμος τερματίζει και επιστρέφει ΨΕΥΔΗΣ.
8: Αν για έστω ένα i δεν υπάρχει ακμή $v_i \to v_{i+1}$, τότε σημαίνει ότι κόμβος v_i και v_{i+1} ανήκουν σε διαφορετικά κλαδιά γράφου.
9: Έτσι, δεν υπάρχει μονοπάτι που να τους συνδέει. Επιστρέφει ΑΛΗΘΗΣ (βρέθηκε μη συγκρίσιμο ζευγάρι).
∗ Ανάλυση Πολυπλοκότητας:
i: Ο DFS επισκέπτεται κάθε κόμβο και κάθε ακμή ακριβώς μία φορά. Χρόνος: $O(n+m)$.
ii: Έπειτα, ψαχνουμε αν ο κόμβος v_{i+1} υπάρχει στη λίστα γειτνίασης του v_i. Ο χρόνος ελέγχου για τον v_i είναι ανάλογος του βαθμού εξόδου του. Αθροίζοντας αυτόν τον χρόνο για όλους τους κόμβους, προκύπτει $O(m)$ συνολικά για όλους τους ελέγχους.
iii: Τελικά, συνολικός Χρόνος: $O(n+m) + O(m) = O(n+m)$, δηλαδή γραμμικός.

• **Πρόβλημα 12ο:**

α.
⋆ Δομή: ένας δισδιάστατος πίνακας M, $n \times n$, όπου $M[u][v] = 1$ εάν υπάρχει μονοπάτι από u στον v, και 0 διαφορετικά.
⋆ Αλγόριθμος κατασκευής: ξεκινάμε BFS ή DFS απο κάθε κόμβο του γράφου για να βρούμε όλους τους προσβάσιμους κόμβους.
⋆ Τελικά: αφού έχουμε τον πίνακα, για οποιοδήποτε ζευγάρι (u, v) η απαντηση δίνεται σε χρόνο $O(1)$, ελέγχοντας τιμή $M[u][v]$.

β.
⋆ ???

γ.
⋆ ???

LaTeX